

NAME : SHAH FAISAL

ROLL NO : 23I-0058

SECTION : BSE – 4A

Operating Systems Lab 08

BSSE SPRING 2026

LAB 08

1. Write a C program that installs signal handlers for SIGINT, SIGTERM, and SIGALRM. When each

signal is received, the program should print a custom message indicating which signal was received. Use pause() to keep the program running and waiting for signals.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler
void handle_signal(int sig) {
    if (sig == SIGINT) {
        printf("Received SIGINT (Ctrl+C)\n");
    }
    else if (sig == SIGTERM) {
        printf("Received SIGTERM\n");
    }
    else if (sig == SIGALRM) {
        printf("Received SIGALRM (Alarm triggered)\n");
    }
}

int main() {
    // Register handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGALRM, handle_signal);

    // Set alarm after 5 seconds
    alarm(5);

    printf("Process PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause(); // Wait for signals
    }

    return 0;
}
```

```
}
```

```
shah@DESKTOP-02Q2MPC:~$ gcc lab8_task1.c -o task1
shah@DESKTOP-02Q2MPC:~$ ./task1
Process PID: 477
Waiting for signals...
Received SIGALRM (Alarm triggered)
Received SIGTERM
Received SIGTERM
```



shah@DESKTOP-02Q2MPC: ~

```
shah@DESKTOP-02Q2MPC:~$ kill -SIGTERM <PID>
-bash: syntax error near unexpected token `newline'
shah@DESKTOP-02Q2MPC:~$ kill -SIGTERM 477
shah@DESKTOP-02Q2MPC:~$ kill -SIGTERM 477
shah@DESKTOP-02Q2MPC:~$
```

2. Write two programs:

Sender Program: This program should take a process ID (PID) as an argument and send a signal

(using kill()) to the specified process. The signal should be specified either by name or by its signal number.

Receiver Program: This program should install a handler for SIGUSR1 and SIGUSR2. When one of

these signals is received, the program should print a message.

Receiver Program

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
```

```
void handle_usr(int sig) {
    if (sig == SIGUSR1) {
        printf("Received SIGUSR1\n");
    }
    else if (sig == SIGUSR2) {
        printf("Received SIGUSR2\n");
    }
}
```

```
int main() {
```

```
signal(SIGUSR1, handle_usr);
signal(SIGUSR2, handle_usr);

printf("Receiver PID: %d\n", getpid());
printf("Waiting for signals...\n");

while (1) {
    pause();
}

return 0;
}
```

Sender Program

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <PID> <signal>\n", argv[0]);
        return 1;
    }

    int pid = atoi(argv[1]);
    int sig;

    // Accept signal name or number
    if (strcmp(argv[2], "SIGUSR1") == 0) {
        sig = SIGUSR1;
    }
    else if (strcmp(argv[2], "SIGUSR2") == 0) {
        sig = SIGUSR2;
    }
    else {
        sig = atoi(argv[2]); // If number is given
    }

    kill(pid, sig);
    printf("Signal sent successfully\n");
}
```

```
return 0;  
}
```

shah@DESKTOP-02Q2MPC: ~

```
shah@DESKTOP-02Q2MPC:~$ nano lab8_task2.c  
shah@DESKTOP-02Q2MPC:~$ nano lab8_task2_1.c  
shah@DESKTOP-02Q2MPC:~$ gcc lab8_task2.c -o task2  
shah@DESKTOP-02Q2MPC:~$ ./task2  
Receiver PID: 521  
Waiting for signals...  
Received SIGUSR1  
Received SIGUSR2  
Received SIGUSR1  
Received SIGUSR2
```

shah@DESKTOP-02Q2MPC: ~

```
shah@DESKTOP-02Q2MPC:~$ ./task21 521 SIGUSR1  
Signal sent successfully  
shah@DESKTOP-02Q2MPC:~$ ./task21 521 SIGUSR2  
Signal sent successfully  
shah@DESKTOP-02Q2MPC:~$
```

3. Write a parent-child program where the parent process forks a child process. The parent should install a handler for SIGCHLD to be notified when the child process terminates. Once the child finishes, the parent should print a message indicating that the child process has completed.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <signal.h>  
#include <sys/wait.h>
```

```
// Handler for SIGCHLD  
void handle_sigchld(int sig) {
```

```
int status;
pid_t pid = wait(&status);

printf("Child process %d completed\n", pid);
}

int main() {
    signal(SIGCHLD, handle_sigchld);

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child (PID: %d) running...\n", getpid());
        sleep(3);
        printf("Child exiting...\n");
        exit(0);
    }
    else if (pid > 0) {
        // Parent process
        printf("Parent (PID: %d) waiting...\n", getpid());

        while (1) {
            pause(); // Wait for SIGCHLD
        }
    }
    else {
        perror("fork failed");
        return 1;
    }

    return 0;
}
```



```
shah@DESKTOP-02Q2MPC:~$ nano lab8_task3.c
shah@DESKTOP-02Q2MPC:~$ gcc lab8_task3.c -o task3
shah@DESKTOP-02Q2MPC:~$ ./task3
Parent (PID: 564) waiting...
Child (PID: 565) running...
Child exiting...
Child process 565 completed
```